

On this page

ETL process from Pulse RealTime (RT) to DataScience Environment (DSE)

The goal of DS Data Migration is to make available the events processed in RT to the DSE environment.

This interaction is represented in the following diagram:

Usage^â

To run the event storage ETL processor we provide two ansible playbooks, `etl.yml` and `etl-single-run.yml`.

See [Feedzai Ansible](#) to understand how to run these playbooks and the playbook [configuration reference](#).



Note that for `etl-single-run` playbook the `tag` is still `etl.install`.

In truth, the playbook is simply an abstraction to empty the `cron_jobs` variable (`cron_jobs: []`).

As an alternative, the variable can be emptied "manually" on the customer defined variables, before running the `etl` playbook.

ETL tool should be installed in a dedicated GCE (Google Cloud Engine) instance using the `etl.yml` playbook.

This playbook sets up a cron job to recurrently migrate a [relative extraction interval](#) of past data.

More precisely, the [default configuration](#) sets up a daily extraction of the data from the previous day.

The GCE instance should have the following permissions:

- Read RT Event Storage (Google Cloud SQL)
- Write to DSE data bucket (Google Cloud Storage bucket)

These accesses can be controlled via both [firewall rules](#) and [service account permissions](#). Details on those topics are outside of the scope of this document.

Extraction Interval^â

Relative extraction interval^â

As explained before, since most commonly ETL runs in a scheduled manner, relative intervals are necessary to actually migrate different data.

This can be achieved with the `-rei` or `--relativeextractioninterval` flags that computes the timestamps following [JTimeWindow Expressions](#).

You can use the [default configuration](#) as reference on how/where to use it.

Fixed extraction interval^â

It is also possible to run ETL for a specific time interval, which can be defined using the following ansible variable / structure:

```
etl_extraction_interval:
start event timestamp: <unix-timestamp>
end event timestamp: <unix-timestamp>
```

Copy



When recovering ETL extractions, make sure to follow the start and end timestamp restrictions described in the [ETL Recovery Procedure](#)



The previous variables will only be considered if `cron_jobs` is not defined or an empty list.

Logging and testing🔗

The default `logger.xml` is provided in `conf/logger.xml`. By default the execution logs will be redirected to:

```
<installation_dir>/log/etl.log
```

Copy

In the previous log file you will be able to find periodic `âReportsâ` about the status of the execution of the data migration about the number of events extracted, and published.

When the extraction finishes a log with the migration result is displayed, similar to the following one:

```
2023-01-23 19:45:17,897 [main] ExecutionMetricFactory.report:330 Report:
[extracted: 8640000/0 @ 4880tps, filtered: 0, published: 8640000/0 @ 4880tps, queued: 0, elapsed
time: 1770532155051 ns]
2023-01-23 19:45:17,898 [main] EventStorageEtlProcessor.runProcess:251 Finished execution
of the ETL process. Time elapsed: 1770532 milliseconds
```

Copy

Exceptions on the data migration will also be written to the previous log file.

Validations:

- After the execution of the tool the events extracted should be equal to the number of the events in the event storage for the same period
- The number of events extracted and published should be the same
- The log file should not have any exceptions
- The file written to the configured bucket should have the number of events published reported (and consequently the same number of events in the event storage for the extraction period)

To check the number of events in event storage:

```
SELECT count(*) FROM eventstorage."7b336ee2c9d8ac105d742c45dbde75" where âFDZ_timestampâ >=
<start timestamp ms> and âFDZ_timestampâ < <end timestamp ms>;
```

Copy